

Begonnen am	Dienstag, 21. Januar 2025, 21:37
Status	Beendet
Beendet am	Freitag, 24. Januar 2025, 12:24
Verbrauchte Zeit	2 Tage 14 Stunden

Information

Hinweise zu den Übungen

Mit den Übungen können Sie das erlernte Wissen und Können festigen.

Gegebenenfalls gibt es "Advanced" Fragen, welche zur weiteren Vertiefung für interessierte Studierende dienen. Diese Aufgaben sind nicht Teil des Prüfungsstoffes im engeren Sinn und als "Advanced" gekennzeichnet..

Bei den meisten Fragen können Sie direkt prüfen, ob Ihre Antwort richtig ist. Gewisse Fragen jedoch (z.B. die offenen Fragen) bieten diese Möglichkeit nicht, weil für diese keine automatisierte Prüfung möglich ist.

Nach Abschluss der gesamten Übung bekommen Sie zusätzlich zu Ihren Resultaten bei etlichen Aufgaben allgemeine Hinweise über die Lösung.

Frage 1

Nicht
beantwortet

Nicht bewertet

Increasing System Performance by Pipelining

Increasing the system performance by instruction pipelining has some considerable increase in complexity and cost in implementation.

What stands on the other side of the balance: How is pipelining achieved and how does this increase the system performance? Is an improvement guaranteed in any case?

Pipelining aims to let all involved HW parts of the CPU that process instruction be busy as much as possible (compared to let only one of each work while the others are idle until their turn in the processing sequence of every one instruction).

If in a given architecture, the pipelining parts (roughly Fetch, Decode, Execute) have similarly long execution times, the increase in system performance is remarkable in a factor of 2-3:

Without pipelining, in this simple model, a complete instruction is processed only after all stages are processed and only then the next instruction can be processed.

With pipelining, after "filling the pipe", with each single pipeline *stage* delay (roughly 1/3 of the overall delay), the next instruction can be processed.

An increase in system performance is not guaranteed due to some potential hazards that may occur in certain situations, such as control hazards (speculation, what is the next instruction to fetch as result of a branch instruction while filling the pipeline) and data hazards (data not yet ready at the point where a following instruction would need it in its current stage). These hazards may cause the pipeline to waste time since the pipeline content might be flushed due to wrong branch speculation or stall until the data is ready.

Frage 2

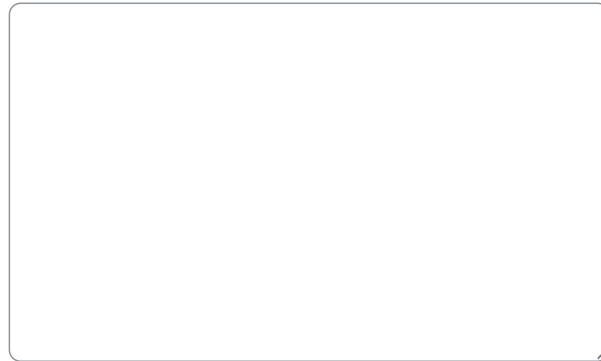
Nicht
beantwortet

Nicht bewertet

Increasing System Performance by RISC architecture

Increasing the system performance by RISC has some impact on the architecture.

What benefit does RISC give over CISC, what stands on the negative side of RISC?



RISC aims to make all instructions similarly simple regarding machine code size per instruction and regarding execution time.

I.e. most of the instructions execute in ideally one CPU clock cycle. This allows to implement simpler pipelining HW and simpler Fetch, Decode, Execute HW blocks. Such simpler pipeline implementation may result in faster pipeline execution speed.

A drawback might be, that the assembly code is less expressive, i.e. for processing data or for programming control flow, more (simpler) instructions have to be used, and therefore, more instructions need to be fetched and processed which might decrease system performance again. When implementing RISC together with pipelineing, a good banalce of these parameters is crucial.

Frage 3

Nicht

beantwortet

Nicht bewertet

Increasing System Performance by Branch Prediction - **Advanced**

Pipelining has the problem of speculative filling the pipe when branch instructions are involved. A condition might not yet be calculated at the time when the next instruction is to be selected to fill into the pipeline, or if the target address of a branch instruction is hold in a register that is not yet loaded.

How does branch prediction try to improve this hazard? Is an increase in system performance guaranteed?

The simplest way of "predicting" what instruction follows the branch instruction (branch taken yes/no?, etc.) is to statically assume that the conditional branch is not taken. For "if/else" control flow, the decision might be arbitrary. On the other hand, for loops, we can assume that they most of the time do actually loop, i.e. the branch to the start of the loop body might be the normal situation - the conditional branch jumps and control flow does not continue with the instruction just after the branch instruction in the memory.

So, a branch prediction machinery could be implemented by "remembering" the last decision made (by taking the first time an arbitrary decision, though). This adds complexity to HW implementation since the information need to be stored and looked up somewhere.

No guarantee for any improvement, but chances for an improvement are high under the assumption that in most of the programs, loops most of the time actually loop multiple times.

Frage 4

Nicht
beantwortet

Nicht bewertet

Further approaches to increase system performance - **Advanced**

What other approaches (beside pipelining and associated improvements such as RISC and branch prediction) "promise" improvements of the system performance.

See the lectures.

- instruction prefetch
- out-of-order execution
- compiler optimizations such as loop unfolding and loop fusion, etc.
- etc.

Frage 5

Nicht

beantwortet

Nicht bewertet

Impact of Pipelining

Lets assume three HW blocks with the following individual execution times

- Fetch: 4 ns
- Decode: 3 ns
- Execute: 5 ns

a) How many instructions per second result **without** pipelining?

b) How many instructions per second (max) can be assumed **with** pipelining?

Please give evidence of the way you calculate the values.

a) **Without** pipelining:

- Instructions per second = $1 / (\text{Fetch} + \text{Decode} + \text{Execute}) = 1 / ((4 + 3 + 5) \text{ ns per Instruction}) = 1 / (12 \times 10^{-9} \text{ sec per Instruction}) = 83.3 \text{ mega instructions per second}$

b) **With** pipelining:

- Instructions per second = $1 / \max(\text{Fetch} + \text{Decode} + \text{Execute}) = 1 / (5 \text{ ns per Instruction}) = 1 / (5 \times 10^{-9} \text{ sec per Instruction}) = 200.0 \text{ mega instructions per second}$